

MSIN0143: Programming for Business Analytics

Type *Markdown* and LaTeX: α^2

Exploratory Analysis on factors that influence Singaporean property prices

Group members:

Kai Bermingham, Student ID: 20097520

Nonthapat Arparpardh, Student ID: 21115118

Xinyu Shen, Student ID: 21026520

Ziqian Li, Student ID:

Table of Content:

- [Section 1: Introduction](#)
 - [1.1 Background](#)
 - [1.2 Problem Statement](#)
 - [1.3 Objective](#)
 - [1.4 Methodology](#)
- [Section 2: Data Preparation](#)
 - [2.1: Data Description](#)
 - [2.2: Data Preparation](#)
 - [2.3: Data Exploration](#)
 - [2.4: Data Dictionary](#)
- [Section 3: Data Cleaning](#)
 - [3.1: Flat Type Cleaning](#)
 - [3.2: Storey Range Cleaning](#)
 - [3.3: Remaining Lease Cleaning](#)
 - [3.4: Removing Null Values](#)
 - [3.5: Comparing Cleaned Dataset with Original Dataset](#)
- [Section 4: Data Analysis](#)
 - [4.1: Distribution of Resale Prices](#)
 - [4.2: Number of Stations in Radius vs Average Resale Price](#)
 - [4.3: Number of Supermarkets in Radius vs Resale Price](#)
 - [4.4: Internal properties of flats \(Floor Area, Flat Type, Remaining Lease\) vs Resale Prices](#)
 - [4.5: Spatial Distribution of Resale Prices](#)
- [Section 5: Prediction Models](#)
 - [5.1: Linear Regression](#)
 - [5.2: Decision Tree Model](#)
- [Section 6: Comparison and Analysis](#)
- [Section 7: References](#)
- [Section 8: Appendices](#)

Section 1: Introduction

1.1 Background:

Singapore is a densely packed city-state in Southeast Asia with a high population living on its small island. To manage its population density and boost home ownership, the government started a housing subsidy program in the 1960s through the Housing & Development Board (HDB). This strategic move has been remarkably effective, with over 80% of residents now living in HDBs, and the total home ownership rate being one of the highest in the world at 89.2% (*Housing & Development Board, 2023*).

1.2 Problem Statement:

Despite their success, the affordability of housing has become a concern. The price-to-income ratio, a key affordability indicator, has risen from 3.33 to 4.07 for new HDBs and 3.98 to 4.23 for resale HDBs between 2001 and 2021 (*Yue, 2023*). This trend, alongside a surge in premium HDB flats surpassing a million dollars, has increased worry for first-time buyers newly entering the market. Studies reveal that these first time buyers are prone to overpaying due to their inexperience, which includes underdeveloped negotiation skills and a lack of market knowledge (*Shui & Murthy, 2019*). Therefore, it is crucial for prospective young buyers to thoroughly examine the various factors that dictate housing prices to avoid overinvestment.

1.3 Objective:

The aim of this report is to investigate the most influential factors that affect the price of Singapore's HDBs, in order to assist first time buyers to assess fair pricing and make well-informed purchasing decisions.

1.4 Methodology:

1. Gathering historical records of HDB resale prices in Singapore, as well as pertinent internal factors such as property features and external factors including public transport links and proximity to shopping centers.
2. Conducting a thorough analysis and visualization of data to identify factors that have a strong correlation with resale values.
3. Developing two predictive model (Gradient Boosting Regression Trees and Linear Regression Model) to forecast resale prices and evaluating its precision.

Section 2: Data Preparation

All libraries used with be shown here:

```
In [1]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity="all"
import numpy
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
from PIL import Image
import IPython.display as display
import io
```

2.1 Data Description

This report utilises 3 main datasets for its research, collected from a combination of open data provided by the Singaporean Government, and through a third party source on GitHub. The first dataset contained information regarding the resale prices and descriptions of properties sold in Singapore, collected from 2017 onwards until the end of 2023. The second dataset contained information on registered supermarkets and their locations, and the last dataset contained information on every public transport train station and their relevant location.

Data Overview of Resale Prices Dataset

In [2]:

```
# Load the 'Resale_flat_prices_2017_onwards.xlsx' file
df_flat_prices = pd.read_excel('Resale_flat_prices_2017_onwards.xlsx')
df_flat_prices.info()
df_flat_prices.head()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 166688 entries, 0 to 166687
Data columns (total 15 columns):
Column Non-Null Count Dtype
--- -
0 month 166688 non-null object
1 town 166688 non-null object
2 flat_type 166688 non-null object
3 block 166688 non-null object
4 street_name 166688 non-null object
5 storey_range 166688 non-null object
6 floor_area_sqm 166688 non-null float64
7 flat_model 166688 non-null object
8 lease_commence_date 166688 non-null int64
9 remaining_lease 166688 non-null object
10 resale_price 166688 non-null float64
11 full address 166688 non-null object
12 POSTAL 166626 non-null object
13 LATITUDE 166626 non-null float64
14 LONGITUDE 166626 non-null float64
dtypes: float64(4), int64(1), object(10)
memory usage: 19.1+ MB

Out [2]:

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_model	lease
0	2017-01	ANG MO KIO	2 ROOM	406	ANG MO KIO AVE 10	10 TO 12	44.0	Improved	
1	2017-01	ANG MO KIO	3 ROOM	108	ANG MO KIO AVE 4	01 TO 03	67.0	New Generation	
2	2017-01	ANG MO KIO	3 ROOM	602	ANG MO KIO AVE 5	01 TO 03	67.0	New Generation	
3	2017-01	ANG MO KIO	3 ROOM	465	ANG MO KIO AVE 10	04 TO 06	68.0	New Generation	
4	2017-01	ANG MO KIO	3 ROOM	601	ANG MO KIO AVE 5	01 TO 03	67.0	New Generation	

Data Overview of MRT SG Dataset

```
In [3]: # Load the 'mrtsg.xlsx' file
df_mrt = pd.read_excel('mrtsg.xlsx')
df_mrt.info()
df_mrt.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 189 entries, 0 to 188
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   OBJECTID    189 non-null    int64
1   STN_NAME    189 non-null    object
2   STN_NO      189 non-null    object
3   X           189 non-null    float64
4   Y           189 non-null    float64
5   Latitude    189 non-null    float64
6   Longitude   189 non-null    float64
7   COLOR       189 non-null    object
dtypes: float64(4), int64(1), object(3)
memory usage: 11.9+ KB
```

Out [3]:

	OBJECTID	STN_NAME	STN_NO	X	Y	Latitude	Longitude	COLOR
0	12	ADMIRALTY MRT STATION	NS10	24402.1063	46918.1131	1.440585	103.800998	RED
1	16	ALJUNIED MRT STATION	EW9	33518.6049	33190.0020	1.316433	103.882893	GREEN
2	33	ANG MO KIO MRT STATION	NS16	29807.2655	39105.7720	1.369933	103.849553	RED
3	81	BAKAU LRT STATION	SE3	36026.0821	41113.8766	1.388093	103.905418	OTHERS
4	80	BANGKIT LRT STATION	BP9	21248.2460	40220.9693	1.380018	103.772667	OTHERS

Data Overview of Supermarkets SG Dataset¶

```
In [4]: df_supermarkets = pd.read_excel('Supermarkets.xlsx')
df_supermarkets.info()
df_supermarkets.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 526 entries, 0 to 525
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Name             526 non-null   object
1   Description       526 non-null   object
2   longitude         526 non-null   float64
3   latitude          526 non-null   float64
dtypes: float64(2), object(2)
memory usage: 16.6+ KB
```

```
Out [4]:
```

	Name	Description	longitude	latitude
0	kml_1		103.901262	1.402303
1	kml_2		103.870914	1.314239
2	kml_3		103.886366	1.373321
3	kml_4		103.914942	1.332959
4	kml_5		103.953010	1.353453

Creating Relevance from MRT and Supermarkets Dataset

The closeness of useful amenities such as shopping and public transport and their impact on housing prices will be studied in this research, thus the Mass Rapid Transit (MRT) and Supermarkets datasets will be combined to the original property price dataset by a common key, location. The radius of 608m was chosen after findings suggested the average Singaporean is willing to walk 608m in their work commute (*Olszewski & Wibowo, 2005*).

```

In [5]: #Determining closest MRT station, color, distance and radius to every
import pandas as pd
from geopy.distance import geodesic #using distance method to quickly
import numpy as np

# Load the 'combined_data_address.xlsx' file
df_combined = pd.read_excel('fetch_data.xlsx')

# Load the 'mrtsg.xlsx' file
df_mrt = pd.read_excel('mrtsg.xlsx')

# Initialize columns for nearest MRT station data and the count within
df_combined['Nearest_MRT_STN_NAME'] = ''
df_combined['Nearest_MRT_COLOR'] = ''
df_combined['Nearest_MRT_Distance'] = np.inf
df_combined['stations_in_radius'] = 0 # New column for count within r

# Iterate over each row in the combined data
for index, row in df_combined.iterrows():
    # Extract latitude and longitude
    lat, lon = row['LATITUDE'], row['LONGITUDE']
    current_location = (lat, lon)

    # Initialize variables to hold the nearest MRT station details
    min_distance = np.inf
    nearest_mrt_name = ''
    nearest_mrt_color = ''

    # Initialize counter for stations within radius
    stations_within_radius = 0

    # Iterate over each MRT station
    for _, mrt_row in df_mrt.iterrows():
        mrt_lat, mrt_lon = mrt_row['Latitude'], mrt_row['Longitude']
        mrt_location = (mrt_lat, mrt_lon)
        distance = geodesic(current_location, mrt_location).kilometers
        # Check if the MRT station is within 0.608 km
        if distance <= 0.608:
            stations_within_radius += 1
        # Check for nearest MRT station
        if distance < min_distance:
            min_distance = distance
            nearest_mrt_name = mrt_row['STN_NAME']
            nearest_mrt_color = mrt_row['COLOR']

    # Update the combined dataframe with the nearest MRT station details
    df_combined.at[index, 'Nearest_MRT_STN_NAME'] = nearest_mrt_name
    df_combined.at[index, 'Nearest_MRT_COLOR'] = nearest_mrt_color
    df_combined.at[index, 'Nearest_MRT_Distance'] = min_distance
    df_combined.at[index, 'stations_in_radius'] = stations_within_radius

# Save the updated dataframe to a new Excel file
df_combined.to_excel('unique_address_and_station_plus_count.xlsx', index=False)

```

```

In [6]: #Finding closest supermarket, and radius to every unique address
import pandas as pd
from geopy.distance import geodesic

# Load the existing data from the Excel files
df_fetched = pd.read_excel('fetched_data.xlsx')
df_supermarkets = pd.read_excel('Supermarkets.xlsx')

# Initialize a new column for counting supermarkets within the radius
df_fetched['Supermarkets_within_608m'] = 0

# Iterate over each row in the fetched data DataFrame
for index, row in df_fetched.iterrows():
    address_coords = (row['LATITUDE'], row['LONGITUDE'])
    supermarkets_within_radius = 0

    # Iterate over each supermarket to count those within 0.608 km
    for _, supermarket in df_supermarkets.iterrows():
        supermarket_coords = (supermarket['latitude'], supermarket['longitude'])
        if geodesic(address_coords, supermarket_coords).kilometers <= 0.608:
            supermarkets_within_radius += 1

    # Update the count in the DataFrame
    df_fetched.at[index, 'Supermarkets_within_608m'] = supermarkets_within_radius

# Save the updated dataframe to a new Excel file called 'unique_address_and_supermarkets.xlsx'
output_file_path = 'unique_address_and_supermarkets.xlsx'
df_fetched.to_excel(output_file_path, index=False)

output_file_path

```

Out[6]: 'unique_address_and_supermarkets.xlsx'


```

In [7]: #Combining every supermarket and MRT info into the full database of sg
import pandas as pd

# Load the existing data from the Excel files
df_combined = pd.read_excel('Resale_flat_prices_2017_onwards.xlsx')
df_unique_mrt = pd.read_excel('unique_address_and_station_plus_count.xlsx')
df_supermarkets = pd.read_excel('unique_address_and_supermarkets.xlsx')

# Initialize new columns for MRT and supermarket information
df_combined['Nearest_MRT_STN_NAME'] = None
df_combined['Nearest_MRT_COLOR'] = None
df_combined['Nearest_MRT_Distance'] = None
df_combined['stations_in_radius'] = None
df_combined['supermarkets_within_608m'] = None

# Iterate over the combined dataframe to update MRT and supermarket info
for index, row in df_combined.iterrows():
    address = row['full address']

    # Update MRT information
    mrt_info = df_unique_mrt[df_unique_mrt['address'] == address]
    if not mrt_info.empty:
        df_combined.at[index, 'Nearest_MRT_STN_NAME'] = mrt_info['Nearest_MRT_STN_NAME'].values[0]
        df_combined.at[index, 'Nearest_MRT_COLOR'] = mrt_info['Nearest_MRT_COLOR'].values[0]
        df_combined.at[index, 'Nearest_MRT_Distance'] = mrt_info['Nearest_MRT_Distance'].values[0]
        df_combined.at[index, 'stations_in_radius'] = mrt_info['stations_in_radius'].values[0]

    # Update supermarket information
    supermarket_info = df_supermarkets[df_supermarkets['address'] == address]
    if not supermarket_info.empty:
        df_combined.at[index, 'supermarkets_within_608m'] = supermarket_info['count'].values[0]

# Save the updated dataframe to a new Excel file
df_combined.to_excel('full_combined_address_mrt_supermarkets.xlsx', index=False)

```

2.2 Data Preparation

To explore the factors and its influence on Singaporean property prices, we combined the 3 datasets into 1 single file

```
In [8]: # Path to the CSV file
file_path = "full_combined_address_mrt_supermarkets.xlsx"

# Read the CSV file with a different encoding
data = pd.read_excel(file_path)
pd.set_option('display.max_columns', None)
data
```

Out [8]:

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_
0	2017-01	ANG MO KIO	2 ROOM	406	ANG MO KIO AVE 10	10 TO 12	44.0	Im
1	2017-01	ANG MO KIO	3 ROOM	108	ANG MO KIO AVE 4	01 TO 03	67.0	Gen
2	2017-01	ANG MO KIO	3 ROOM	602	ANG MO KIO AVE 5	01 TO 03	67.0	Gen
3	2017-01	ANG MO KIO	3 ROOM	465	ANG MO KIO AVE 10	04 TO 06	68.0	Gen
4	2017-01	ANG MO KIO	3 ROOM	601	ANG MO KIO AVE 5	01 TO 03	67.0	Gen
...	...	...	...	...	...	...	...	...
166683	2023-05	YISHUN	EXECUTIVE	827	YISHUN ST 81	01 TO 03	154.0	Mais
166684	2023-05	YISHUN	EXECUTIVE	824	YISHUN ST 81	07 TO 09	146.0	Mais
166685	2023-07	YISHUN	EXECUTIVE	837	YISHUN ST 81	01 TO 03	146.0	Mais
166686	2023-01	YISHUN	MULTI-GENERATION	666	YISHUN AVE 4	07 TO 09	179.0	Gen
166687	2023-06	YISHUN	MULTI-GENERATION	633	YISHUN ST 61	04 TO 06	159.0	Gen

166688 rows × 20 columns

In [11]: data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 166688 entries, 0 to 166687
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   month                                166688 non-null  object
1   town                                166688 non-null  object
2   flat_type                            166688 non-null  object
3   block                                166688 non-null  object
4   street_name                          166688 non-null  object
5   storey_range                         166688 non-null  object
6   floor_area_sqm                       166688 non-null  float64
7   flat_model                           166688 non-null  object
8   lease_commence_date                 166688 non-null  int64
9   remaining_lease                     166688 non-null  object
10  resale_price                         166688 non-null  float64
11  full address                         166688 non-null  object
12  POSTAL                              166626 non-null  object
13  LATITUDE                            166626 non-null  float64
14  LONGITUDE                           166626 non-null  float64
15  Nearest_MRT_STN_NAME                166626 non-null  object
16  Nearest_MRT_COLOR                   166626 non-null  object
17  Nearest_MRT_Distance                 166626 non-null  float64
18  stations_in_radius                  166626 non-null  float64
19  supermarkets_within_608m            166626 non-null  float64
dtypes: float64(7), int64(1), object(12)
memory usage: 25.4+ MB
```

2.3 Data Exploration

To ensure that there are sufficient data and attributes to analyze, we can print the dimensions of the dataset

In [12]: `print('The dataset has {0} rows and {1} columns' .format(data.shape[0]`

The dataset has 166688 rows and 20 columns

2.4 Data Dictionary

```
In [13]: description = ["Date Listed", "Located Town", "Flat type", "Located Block Number",
                        "Date when flat lease started", "Remaining Lease in Years and Months",
                        "Color of Flat's nearest MRT line", "Distance between Flat and Nearest MRT Station",
                        "Flat Resale Price in SGD"]
example = ["2019-08", "BUKIT TIMAH", "3 ROOM", "255", "ANG MO KIO AVE 10", "07 TO 09",
           "1.37405846295585", "103.835475719134", "BEDOK NORTH MRT STATION"]
data_dict={'Column Name':data.columns,'Description':description,'Example of data':example}
data_dictionary=pd.DataFrame(data_dict)
pd.set_option('display.max_colwidth',None)
data_dictionary
```

Out [13]:

	Column Name	Description	Example of data
0	month	Date Listed	2019-08
1	town	Located Town	BUKIT TIMAH
2	flat_type	Flat type	3 ROOM
3	block	Located Block Number	255
4	street_name	Street Name	ANG MO KIO AVE 10
5	storey_range	Flat Storey Range	07 TO 09
6	floor_area_sqm	Flat Floor Area in SQM	92
7	flat_model	Flat Model	New Generation
8	lease_commence_date	Date when flat lease started	1978
9	remaining_lease	Remaining Lease in Years and Months	59 years 06 months
10	resale_price	Full Address	536 ANG MO KIO AVE 10

Section 3: Data Cleaning

To clean the dataset, a duplicated version of the dataset is used in order to maintain the original version prior to cleaning

```
In [14]: data_to_clean = data.copy()
original_data = data
```

3.1 Flat Type Cleaning

```
In [15]: import pandas as pd
from IPython.display import display

# Function to convert 'flat_type' values to numbers
def convert_flat_type(flat_type):
    mapping = {'1 ROOM': 1, '2 ROOM': 2, '3 ROOM': 3, '4 ROOM': 4, '5 ROOM': 5}
    return mapping.get(flat_type, None) # Returns None if the flat type is not in the mapping

# Apply the conversion function to 'flat_type' column
data_to_clean['flat_type'] = data_to_clean['flat_type'].apply(convert_flat_type)

# Create a new DataFrame for comparison that includes the original and cleaned data
comparison_df = pd.DataFrame({
    'Before Cleaning': original_data['flat_type'],
    'After Cleaning': data_to_clean['flat_type']
})

# Display the comparison DataFrame
display(comparison_df)
```

	Before Cleaning	After Cleaning
0	2 ROOM	2
1	3 ROOM	3
2	3 ROOM	3
3	3 ROOM	3
4	3 ROOM	3
...	...	...
166683	EXECUTIVE	6
166684	EXECUTIVE	6
166685	EXECUTIVE	6
166686	MULTI-GENERATION	7
166687	MULTI-GENERATION	7

166688 rows × 2 columns

3.2: Storey range Cleaning

```
In [16]: # Function to find the midpoint of the 'storey_range'
def convert_storey_range(storey_range):
    low, high = storey_range.split(' TO ')
    return (int(low) + int(high)) // 2

data_to_clean['storey_range'] = data_to_clean['storey_range'].apply(cc

# Create a new DataFrame for comparison that includes the original and
comparison_df = pd.DataFrame({
    'Before Cleaning': original_data['storey_range'],
    'After Cleaning': data_to_clean['storey_range']
})
# Display the data to check if the conversion was successful
display(comparison_df)
```

	Before Cleaning	After Cleaning
0	10 TO 12	11
1	01 TO 03	2
2	01 TO 03	2
3	04 TO 06	5
4	01 TO 03	2
...	...	...
166683	01 TO 03	2
166684	07 TO 09	8
166685	01 TO 03	2
166686	07 TO 09	8
166687	04 TO 06	5

166688 rows × 2 columns

3.3: Remaining Lease Cleaning

```
In [17]: # Function to convert 'remaining_lease' to total months
def convert_remaining_lease(lease):
    parts = lease.split(' ')
    years = int(parts[0]) # The first part is always the number of years
    months = int(parts[2]) if 'month' in parts else 0 # The third part is the number of months
    return years * 12 + months

data_to_clean['remaining_lease'] = data_to_clean['remaining_lease'].apply(convert_remaining_lease)

# Create a new DataFrame for comparison that includes the original and cleaned data
comparison_df = pd.DataFrame({
    'Before Cleaning': original_data['remaining_lease'],
    'After Cleaning': data_to_clean['remaining_lease']
})

# Display the data to check if the conversion was successful
display(comparison_df)
```

	Before Cleaning	After Cleaning
0	61 years 04 months	732
1	60 years 07 months	720
2	62 years 05 months	744
3	62 years 01 month	745
4	62 years 05 months	744
...	...	...
166683	63 years 06 months	756
166684	63 years 06 months	756
166685	63 years 07 months	756
166686	63 years 11 months	756
166687	63 years 06 months	756

166688 rows × 2 columns

3.4: Removing Null Values

```
In [18]: # Remove rows with any NaN values in any column
data_to_clean = data_to_clean.dropna()
```

3.5: Comparing Cleaned Dataset with Original Dataset

In [19]:

display(original_data)
display(data_to_clean)

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_
0	2017-01	ANG MO KIO	2 ROOM	406	ANG MO KIO AVE 10	10 TO 12	44.0	Im
1	2017-01	ANG MO KIO	3 ROOM	108	ANG MO KIO AVE 4	01 TO 03	67.0	Gen
2	2017-01	ANG MO KIO	3 ROOM	602	ANG MO KIO AVE 5	01 TO 03	67.0	Gen
3	2017-01	ANG MO KIO	3 ROOM	465	ANG MO KIO AVE 10	04 TO 06	68.0	Gen
4	2017-01	ANG MO KIO	3 ROOM	601	ANG MO KIO AVE 5	01 TO 03	67.0	Gen
...	...	...	...	...	...	...	...	...
166683	2023-05	YISHUN	EXECUTIVE	827	YISHUN ST 81	01 TO 03	154.0	Mais
166684	2023-05	YISHUN	EXECUTIVE	824	YISHUN ST 81	07 TO 09	146.0	Mais
166685	2023-07	YISHUN	EXECUTIVE	837	YISHUN ST 81	01 TO 03	146.0	Mais
166686	2023-01	YISHUN	MULTI-GENERATION	666	YISHUN AVE 4	07 TO 09	179.0	Gen
166687	2023-06	YISHUN	MULTI-GENERATION	633	YISHUN ST 61	04 TO 06	159.0	Gen

166688 rows × 20 columns

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_mod
0	2017-01	ANG MO KIO	2	406	ANG MO KIO AVE 10	11	44.0	Improve
1	2017-01	ANG MO KIO	3	108	ANG MO KIO AVE 4	2	67.0	Ne Generatic
2	2017-01	ANG MO KIO	3	602	ANG MO KIO AVE 5	2	67.0	Ne Generatic
3	2017-01	ANG MO KIO	3	465	ANG MO KIO AVE 10	5	68.0	Ne Generatic
4	2017-01	ANG MO KIO	3	601	ANG MO KIO AVE 5	2	67.0	Ne Generatic
...	...	...	...	...	...	...	...	...
166683	2023-05	YISHUN	6	827	YISHUN ST 81	2	154.0	Maisonet
166684	2023-05	YISHUN	6	824	YISHUN ST 81	8	146.0	Maisonet
166685	2023-07	YISHUN	6	837	YISHUN ST 81	2	146.0	Maisonet
166686	2023-01	YISHUN	7	666	YISHUN AVE 4	8	179.0	Mu Generatic
166687	2023-06	YISHUN	7	633	YISHUN ST 61	5	159.0	Mu Generatic

166626 rows × 20 columns

Section 4: Data Analysis

```
In [20]: data = data_to_clean.copy()
from matplotlib.ticker import FuncFormatter
import matplotlib.ticker as ticker
from PIL import Image
```

```
In [21]: numerical_cols = [
        'block', 'storey_range', 'floor_area_sqm', 'lease_commence_date', 'remaining_lease',
        'flat_type', 'LATITUDE', 'LONGITUDE', 'Nearest_MRT_Distance', 'stations_in_radius',
        'supermarkets_within_608m', 'resale_price'
    ]

# Attempt to convert only the specified columns to numeric and drop any non-numeric columns
df_numerical = data[numerical_cols].apply(pd.to_numeric, errors='coerce')

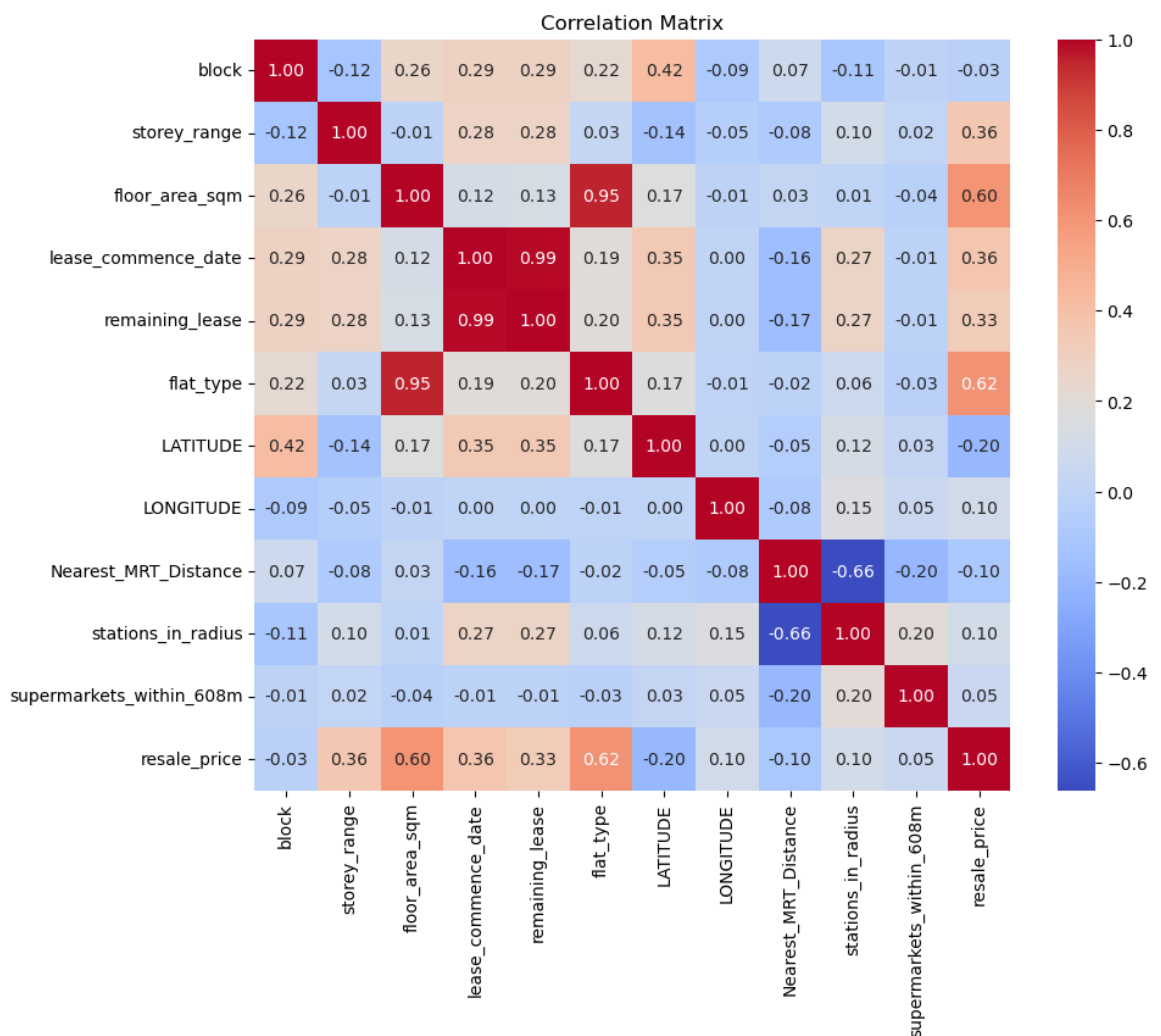
# Recalculate the correlation matrix with only the numeric columns
correlation_matrix = df_numerical.corr()

# Create the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

Out[21]: <Figure size 1000x800 with 0 Axes>

Out[21]: <AxesSubplot:>

Out[21]: Text(0.5, 1.0, 'Correlation Matrix')



Findings: The heatmap indicates that the size of the living space (**floor_area_sqm**) and the type of housing unit (**flat_type**) have a relatively strong positive correlation with the resale prices of homes. The proximity to amenities like the number of nearby subway

stations (stations_in_radius) and supermarkets within 608 meters (supermarkets_within_608m) also show a positive correlation with resale prices, although to a lesser extent.

The variable "block" was not further researched due to the non-existence of a correlation, indicated by the smallest r-value of -0.03.

4.1: Distribution of Resale Prices

```
In [23]: sns.set(style="whitegrid")
plt.figure(figsize=(10, 6))

sns.histplot(data['resale_price'], bins=50, kde=True)

plt.title('Distribution of Resale Prices')
plt.xlabel('Resale Price')
plt.ylabel('Frequency')

formatter = FuncFormatter(lambda x, _: f'{int(x/1000)}K')
plt.gca().xaxis.set_major_formatter(formatter)

plt.show()
```

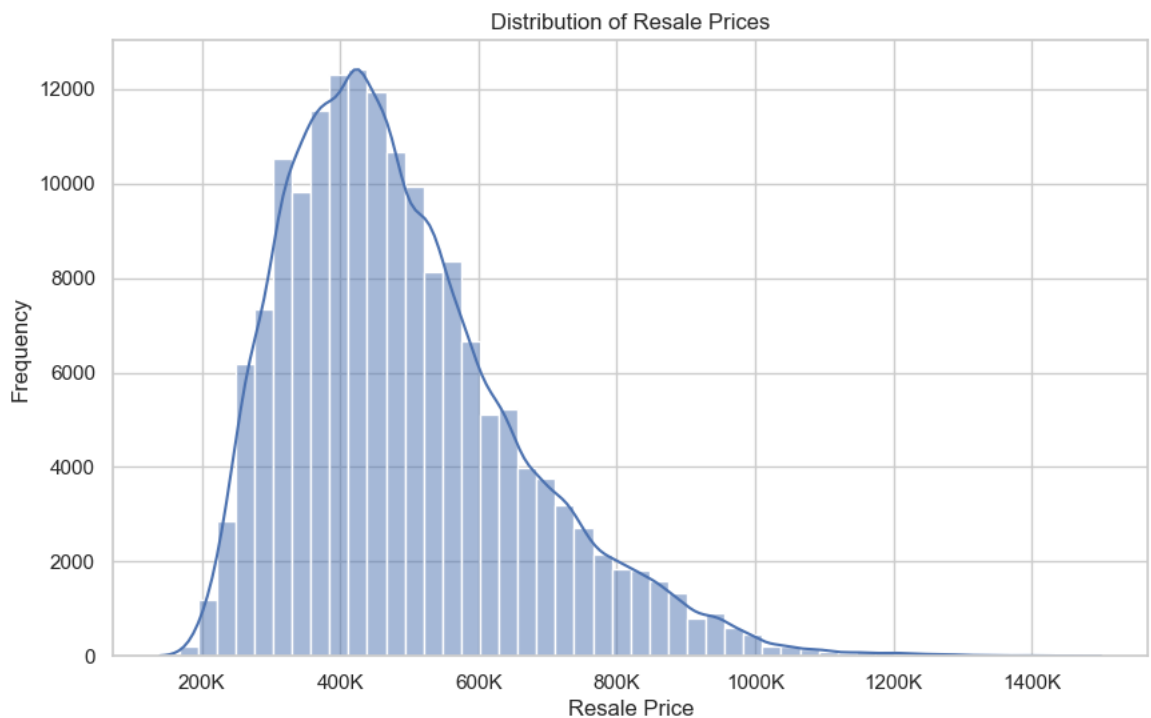
Out[23]: <Figure size 1000x600 with 0 Axes>

Out[23]: <AxesSubplot:xlabel='resale_price', ylabel='Count'>

Out[23]: Text(0.5, 1.0, 'Distribution of Resale Prices')

Out[23]: Text(0.5, 0, 'Resale Price')

Out[23]: Text(0, 0.5, 'Frequency')



```
In [24]: # Display basic descriptive statistics for the 'resale_price' column
basic_stats = data['resale_price'].describe()

# Calculate additional descriptive statistics
variance = data['resale_price'].var()
range_value = data['resale_price'].max() - data['resale_price'].min()
Q1 = data['resale_price'].quantile(0.25)
Q3 = data['resale_price'].quantile(0.75)
iqr = Q3 - Q1
skewness = data['resale_price'].skew()
kurtosis = data['resale_price'].kurtosis()

# Print the combined descriptive statistics
print("Combined Descriptive Statistics for 'resale_price' Column:")
print(basic_stats)
print(f'Variance: {variance}')
print(f'Range: {range_value}')
print(f'Interquartile Range (IQR): {iqr}')
print(f'Skewness: {skewness}')
print(f'Kurtosis: {kurtosis}')
```

Combined Descriptive Statistics for 'resale_price' Column:

```
count    1.666260e+05
mean     4.896538e+05
std      1.693773e+05
min      1.400000e+05
25%      3.650000e+05
50%      4.600000e+05
75%      5.800000e+05
max      1.500000e+06
Name: resale_price, dtype: float64
Variance: 28688662019.243816
Range: 1360000.0
Interquartile Range (IQR): 215000.0
Skewness: 0.9508338840570655
Kurtosis: 1.0176472789065372
```

Findings: A descriptive statistics can be derived from the Distribution of Resale Prices. The majority of Singaporean flat resale prices sit at around \$489,669. The distribution is right-skewed with skewness of around 0.95, indicating that the majority of flats are relatively affordable with most resale prices leaning toward the lower end of the price range. Although there are a small number of flats with significantly higher prices.

4.2: Number of Stations in Radius vs Average Resale Price

Does having public transport within walking distance impact the flat's price?

```
In [25]: # Set the style
sns.set(style="whitegrid")

# Create a figure with the second subplot
fig, ax = plt.subplots(figsize=(12, 6))

# Group data by 'stations_in_radius' and calculate mean resale prices
grouped_data = data.groupby('stations_in_radius')['resale_price'].mean()

# Create a bar plot for number of stations in radius vs average resale price
sns.barplot(x='stations_in_radius', y='resale_price', data=grouped_data)
ax.set_title('Number of Stations in Radius vs Average Resale Price')
ax.set_xlabel('Number of Stations in Radius')
ax.set_ylabel('Average Resale Price')

# Show the second graph
plt.show()

data['resale_price'] = (data['resale_price'] / 1000).round() * 1000
sns.set(style="whitegrid")

# Create a figure with a single subplot for the KDE plot
fig, ax = plt.subplots(figsize=(8, 6))
formatter = ticker.StrMethodFormatter('{x:,.0f}')

# KDE plot
sns.kdeplot(data=data, x='Nearest_MRT_Distance', y='resale_price', fill=True)
ax.set_title('Distance to Nearest MRT vs Resale Price')
ax.set_xlabel('Distance to Nearest MRT (meters)')
ax.set_ylabel('Resale Price (Rounded)')
ax.yaxis.set_major_formatter(formatter)

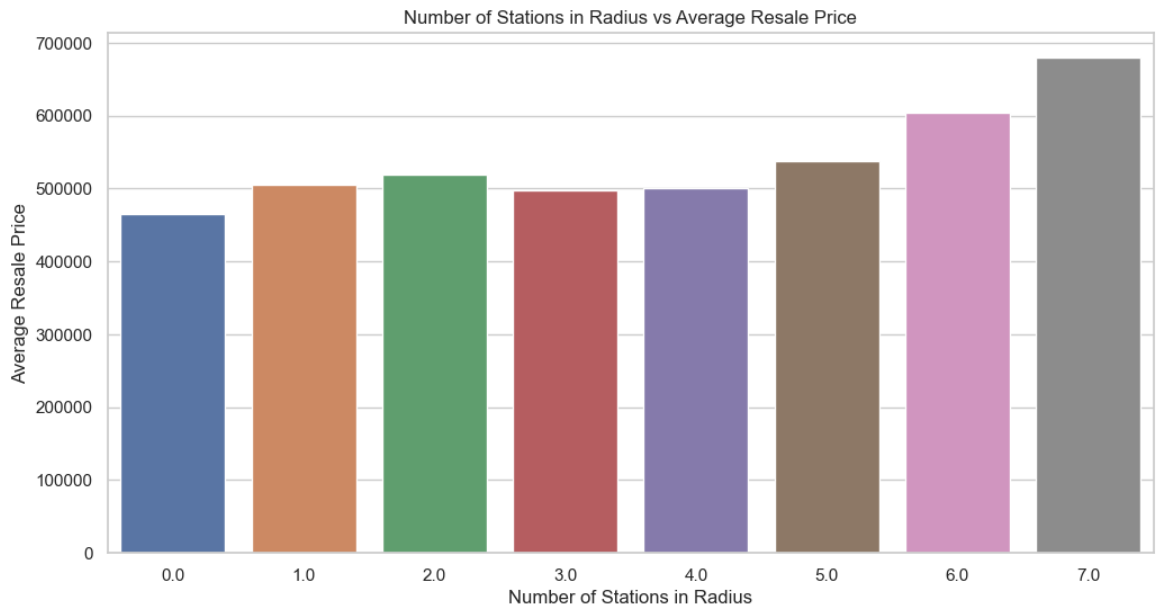
# Adjust layout
plt.tight_layout()
plt.show()
```

Out [25]: <AxesSubplot:xlabel='stations_in_radius', ylabel='resale_price'>

Out [25]: Text(0.5, 1.0, 'Number of Stations in Radius vs Average Resale Price')

Out [25]: Text(0.5, 0, 'Number of Stations in Radius')

Out [25]: Text(0, 0.5, 'Average Resale Price')

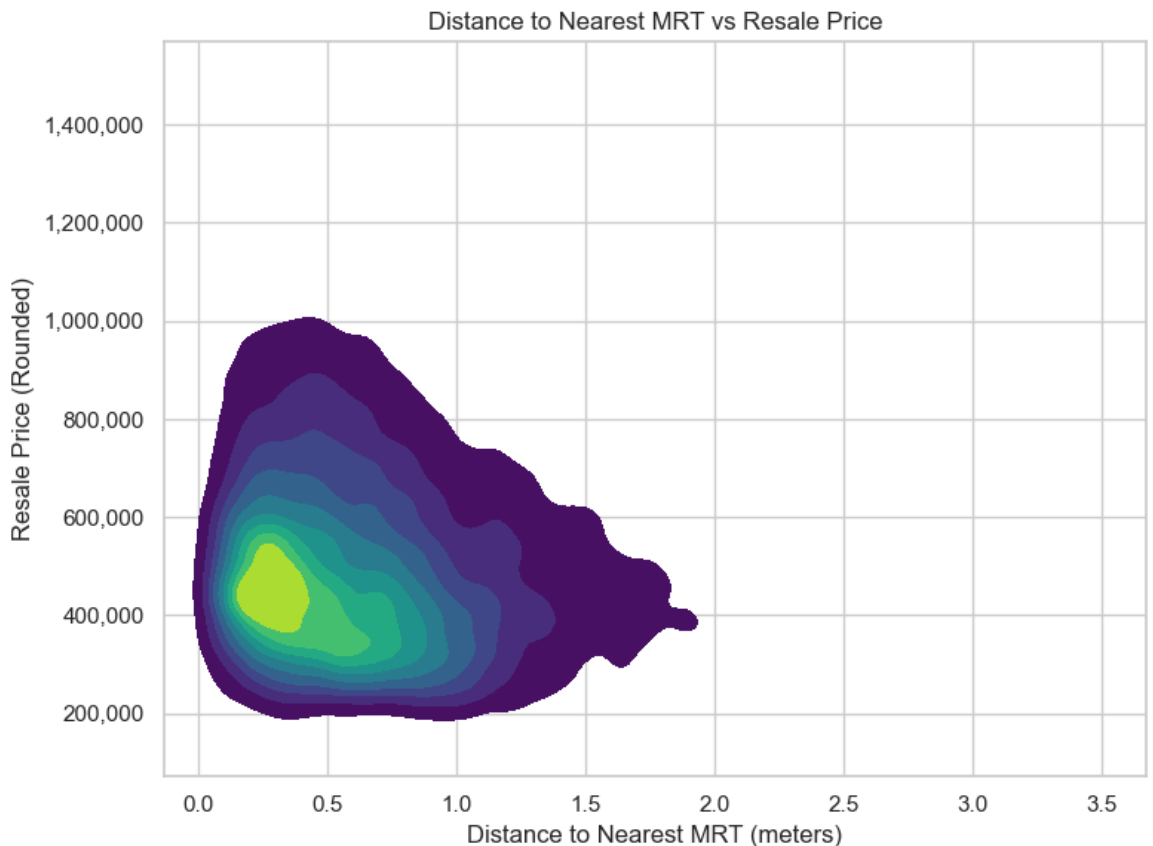


Out[25]: <AxesSubplot:xlabel='Nearest_MRT_Distance', ylabel='resale_price'>

Out[25]: Text(0.5, 1.0, 'Distance to Nearest MRT vs Resale Price')

Out[25]: Text(0.5, 0, 'Distance to Nearest MRT (meters)')

Out[25]: Text(0, 0.5, 'Resale Price (Rounded)')



Findings: The plot exhibits a clear trend where resale prices are highest near MRT stations and decrease as the distance increases. This trend indicates a strong preference for proximity to public transit, significantly influencing property values in Singapore. The concentration of higher resale prices close to MRT stations suggests that accessibility is a key determinant of housing desirability. However, an odd trend is flats with 2 stations have a higher average price than flats with 3 and 4 stations.

```
In [26]: column_name = 'stations_in_radius'

# Count the number of flats for each unique value in the specified column
count_by_value = data[column_name].value_counts().reset_index()

# Rename the columns to add headers
count_by_value.columns = ['Number of Stations', 'Flat Count']

# Display the result
print(count_by_value)
```

	Number of Stations	Flat Count
0	0.0	72124
1	1.0	52239
2	2.0	21335
3	3.0	13193
4	4.0	5601
5	5.0	1606
6	6.0	496
7	7.0	32

Limitation: The anomaly where flats with 2 stations are more expensive than flats with 3 and 4 stations may be influenced by the unequal distribution of flats. Given that the sample size of flats with two stations exceeds almost double those with three and four stations, this discrepancy may have influenced the average resale price.

4.3: Number of Supermarkets in Radius vs Resale Price

Does the number of supermarkets surrounding a flat impact its resale price?

```
In [27]: sns.set(style="whitegrid")

# Creating a single figure for the third subplot
fig, ax = plt.subplots(figsize=(12, 6))

# Plotting the third subplot for supermarkets_within_608m vs resale_price
sns.lineplot(data=data, x='supermarkets_within_608m', y='resale_price')
ax.set_title('Number of Supermarkets within 608m vs Resale Price')
ax.set_xlabel('Number of Supermarkets within 608m')
ax.set_ylabel('Resale Price')

# Adjusting the layout and displaying the plot
plt.tight_layout()
plt.show()
```

Out [27]: <AxesSubplot:xlabel='supermarkets_within_608m', ylabel='resale_price'>

Out [27]: Text(0.5, 1.0, 'Number of Supermarkets within 608m vs Resale Price')

Out [27]: Text(0.5, 0, 'Number of Supermarkets within 608m')

Out [27]: Text(0, 0.5, 'Resale Price')



Findings: There is a positive correlation between number of supermarkets and resale price until number of supermarkets reached 7. Once the number of supermarkets exceeded 7, the resale price decreases, suggesting that a large quantity of supermarkets may result in high traffic congestion and inconvenience, leading to a depletion on resale price

4.4 Internal properties of flats (*Floor Area, Flat Type, Remaining Lease*) vs Resale Prices

Do internal properties of resale flats directly correlate with its resale price?

```
In [28]: sns.set(style="whitegrid")

# Creating a figure with multiple subplots
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 10))
formatter = ticker.StrMethodFormatter('{x:,.0f}')

# Scatterplot with regression line
sns.regplot(data=data, x='floor_area_sqm', y='resale_price', ax=axes[0, 0])
axes[0, 0].set_title('Floor Area vs Resale Price')
axes[0, 0].set_xlabel('Floor Area (sqm)')
axes[0, 0].set_ylabel('Resale Price')
axes[0, 0].yaxis.set_major_formatter(formatter)

# Violin plot
sns.violinplot(data=data, x='flat_type', y='resale_price', ax=axes[0, 1])
axes[0, 1].set_title('Flat Type vs Resale Price')
axes[0, 1].set_xlabel('Flat Type')
axes[0, 1].set_ylabel('Resale Price')
axes[0, 1].yaxis.set_major_formatter(formatter)
axes[0, 1].yaxis.set_major_formatter(formatter)

# Hexbin plot
hexbin = axes[1, 0].hexbin(data['remaining_lease'], data['resale_price'])
cb = fig.colorbar(hexbin, ax=axes[1, 0])
axes[1, 0].set_title('Remaining Lease vs Resale Price')
axes[1, 0].set_xlabel('Remaining Lease (months)')
axes[1, 0].set_ylabel('Resale Price (Rounded)')
axes[1, 0].yaxis.set_major_formatter(formatter)

# To not display the bottom right graph
axes[1, 1].axis('off')

# Adjust layout
plt.tight_layout()
plt.show()
```

Out [28]: <AxesSubplot:xlabel='floor_area_sqm', ylabel='resale_price'>

Out [28]: Text(0.5, 1.0, 'Floor Area vs Resale Price')

Out [28]: Text(0.5, 0, 'Floor Area (sqm)')

Out [28]: Text(0, 0.5, 'Resale Price')

Out [28]: <AxesSubplot:xlabel='flat_type', ylabel='resale_price'>

Out [28]: Text(0.5, 1.0, 'Flat Type vs Resale Price')

Out [28]: Text(0.5, 0, 'Flat Type')

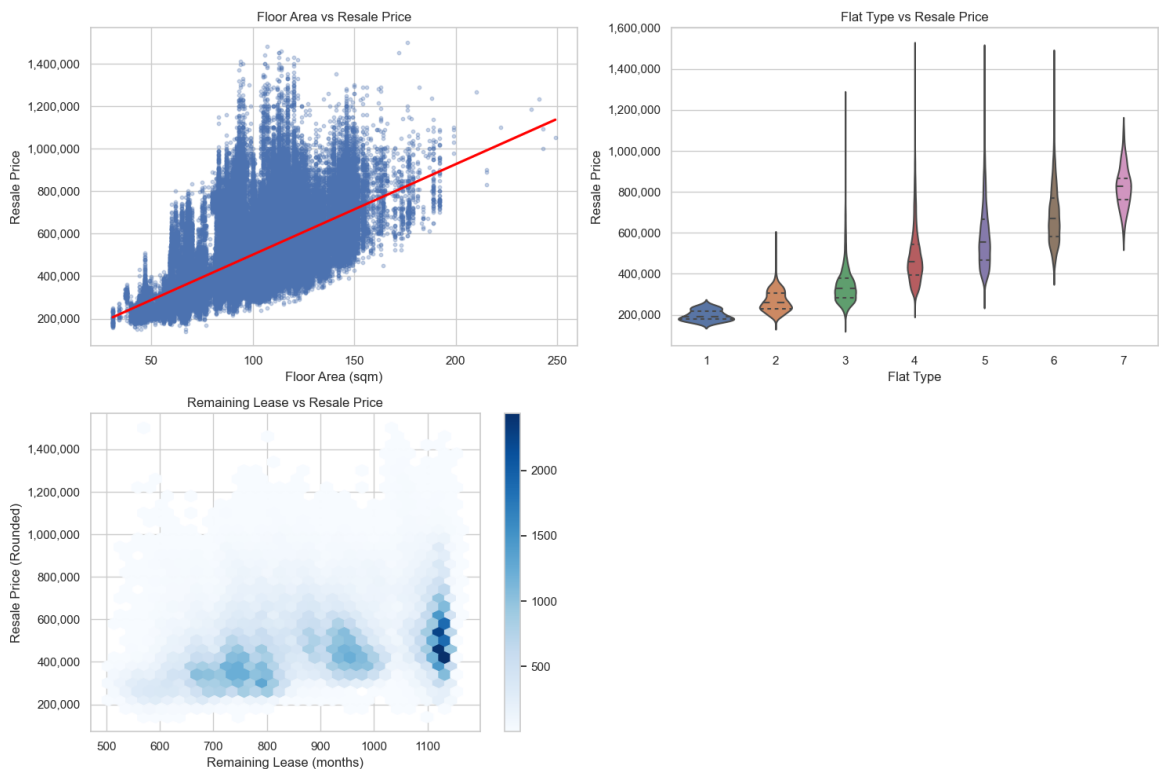
Out [28]: Text(0, 0.5, 'Resale Price')

Out[28]: Text(0.5, 1.0, 'Remaining Lease vs Resale Price')

Out[28]: Text(0.5, 0, 'Remaining Lease (months)')

Out[28]: Text(0, 0.5, 'Resale Price (Rounded)')

Out[28]: (0.0, 1.0, 0.0, 1.0)



How does resale price vary with the floor area of the flats?

Findings: The scatter plot shows a positive relationship between floor area and resale price. Generally, as the floor area increases, the resale price tends to increase as well. While the trend is positive, the relationship is not perfectly linear. This suggests that other factors (like location, age of the building, or nearby amenities) might also influence the resale price.

What is the relationship between different flat types and their resale prices?

Findings: Higher flat types (translating to flats with more rooms) tend to have a higher range of resale prices.

Supported by the median prices for each flat type rising progressively from 1-bedroom to 7-bedroom flats.

Findings: The variability in resale prices is minimal especially for 1-bedroom flats, and arguably for 2-bedroom and the most expensive 7 bed-room flats too, as shown by the compactness of their respective plot points.

This suggests a stable and more predictable pricing for smaller flats, which could be due to a consistent set of features or less variation in other important factors such as size or amenities among these flat types. The compactness of the 7-bedroom flat however, may be explained by the smaller sample size of large properties in the database.

How does the length of the remaining lease affect the resale price?

Findings: The hexbin plot shows a more dispersed relationship. It's less clear how the remaining lease length directly impacts the resale price, suggesting other factors might play a more significant role.

```
In [29]: sns.set(style="whitegrid")
plt.figure(figsize=(14, 7))

# Plot the violin plot for flat_model vs resale_price
violinplot = sns.violinplot(data=data, x='flat_model', y='resale_price')

# Set the title and labels of the plot
violinplot.set_title('Resale Price Distribution by Flat Model')
violinplot.set_xlabel('Flat Model')
violinplot.set_ylabel('Resale Price')

# Format the y-axis to show labels as rounded thousands without scientific notation
violinplot.yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

# Rotate x-axis labels if needed for better readability
plt.xticks(rotation=90)

# Show the plot
plt.show()
```

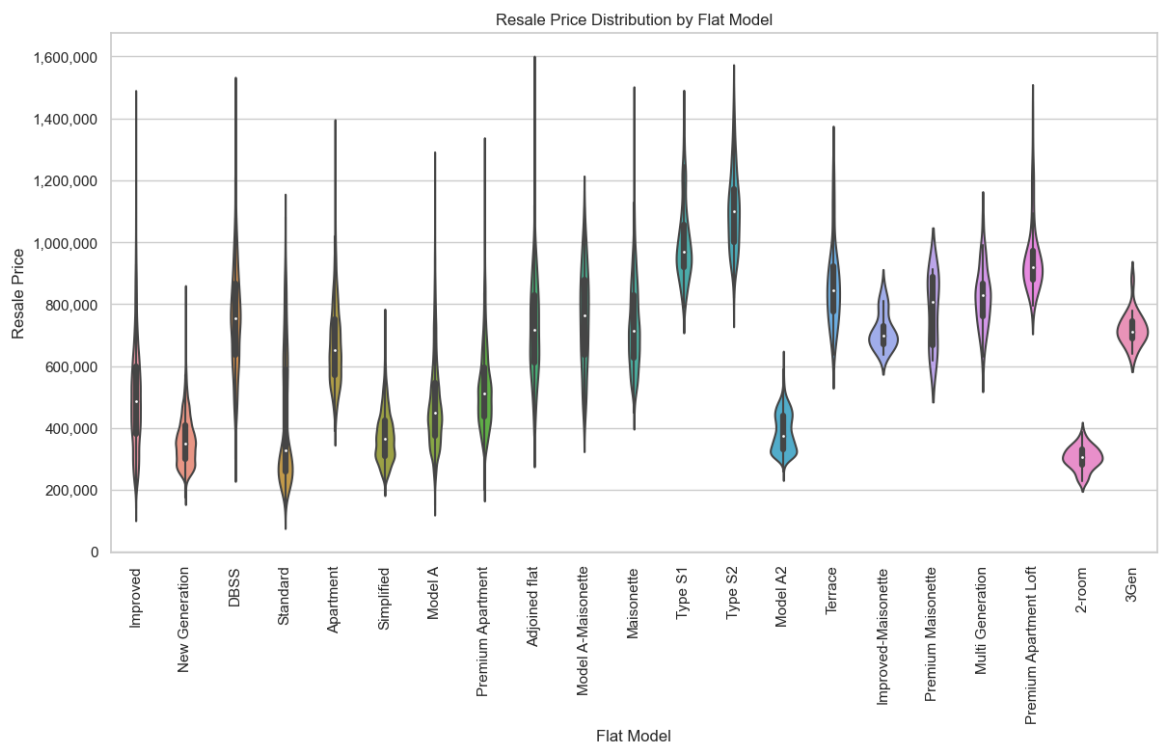
Out [29]: <Figure size 1400x700 with 0 Axes>

Out [29]: Text(0.5, 1.0, 'Resale Price Distribution by Flat Model')

Out [29]: Text(0.5, 0, 'Flat Model')

Out [29]: Text(0, 0.5, 'Resale Price')

Out [29]: (array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]),
 [Text(0, 0, 'Improved'),
 Text(1, 0, 'New Generation'),
 Text(2, 0, 'DBSS'),
 Text(3, 0, 'Standard'),
 Text(4, 0, 'Apartment'),
 Text(5, 0, 'Simplified'),
 Text(6, 0, 'Model A'),
 Text(7, 0, 'Premium Apartment'),
 Text(8, 0, 'Adjoined flat'),
 Text(9, 0, 'Model A-Maisonette'),
 Text(10, 0, 'Maisonette'),
 Text(11, 0, 'Type S1'),
 Text(12, 0, 'Type S2'),
 Text(13, 0, 'Model A2'),
 Text(14, 0, 'Terrace'),
 Text(15, 0, 'Improved-Maisonette'),
 Text(16, 0, 'Premium Maisonette'),
 Text(17, 0, 'Multi Generation'),
 Text(18, 0, 'Premium Apartment Loft'),
 Text(19, 0, '2-room'),
 Text(20, 0, '3Gen')])



How flat model affects the resale price?

Findings:

In the violin plot, 'Adjoined flat' and 'Type S2' types contain the highest resale price models, indicated by the highest peaks, with the 'Type S1', 'Type S2' and 'Premium Apartment Loft' models maintaining the highest average resale price, suggesting a premium for these spacious designs. The '2-room', 'Model A2', and 'Simplified' models reflect lower resale values with low variability, indicating affordability and consistency in the market. Models like 'Premium Apartment' and 'Model A' display a moderate price range, indicative of a middle market segment. Overall, flat model types are a key indicator of resale price, with specific models commanding higher prices.

4.5 Spatial Distribution of Resale Prices

Does the location of the flat affect resale prices?

```
In [32]: import pandas as pd
import folium
import numpy as np
import branca

# Load just the relevant columns
df = data[['LATITUDE', 'LONGITUDE', 'resale_price']].copy()

# Define the bins for the resale_price
bins = np.linspace(df['resale_price'].min(), df['resale_price'].max(),
df['price_bin'] = pd.cut(df['resale_price'], bins=bins, include_lowest

# Define colors for each bin
colors = ['blue', 'green', 'yellow', 'orange', 'red']
bin_colors = pd.cut(df['resale_price'], bins=bins, labels=colors)

# Create a Folium map
m = folium.Map(location=[df['LATITUDE'].mean(), df['LONGITUDE'].mean()])

# Add a circle marker for each property in the DataFrame
for lat, lon, color in zip(df['LATITUDE'], df['LONGITUDE'], bin_colors):
    folium.CircleMarker(
        location=[lat, lon],
        radius=1.5,
        color=color,
        fill=True,
        fill_color=color,
        fill_opacity=0.6
    ).add_to(m)

# Define the HTML for the legend
legend_html = '''
<div style="position: fixed;
    bottom: 25px; left: 25px; width: 175px; height: 150;
    border: 2px solid grey; z-index: 9999; font-size: 14px; background-color: white; padding: 5px;">
    Price Range <br>
    <i style="background-color: blue; border-radius: 50%; width: 20px; height: 10px; display: inline-block; margin-right: 5px;">
    <i style="background-color: green; border-radius: 50%; width: 20px; height: 10px; display: inline-block; margin-right: 5px;">
    <i style="background-color: yellow; border-radius: 50%; width: 20px; height: 10px; display: inline-block; margin-right: 5px;">
    <i style="background-color: orange; border-radius: 50%; width: 20px; height: 10px; display: inline-block; margin-right: 5px;">
    <i style="background-color: red; border-radius: 50%; width: 20px; height: 10px; display: inline-block; margin-right: 5px;">
</div>
'''

# Add the legend to the map
m.get_root().html.add_child(folium.Element(legend_html))

# Display the map
m
```

Out[32]: <folium.vector_layers.CircleMarker at 0x7fd7917c2e20>

```
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd7b4f9cd60>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35940>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35b20>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e358b0>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35970>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35af0>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35850>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35a60>
Out[32]: <folium.vector_layers.CircleMarker at 0x7fd723e35910>
```

Finding 1: Highest concentration of listings appear to be in outer regions such as East, North-east, and Northern regions, sparser distribution appear in Central regions particularly in towns like Bukit Timah, Pasir Panjang, and Orchard.

Anomaly of lack of properties in Bukit Timah area can be explained by high concentration of private property developments, like luxury Good Class Bungaloes (Sng, 2022)

Finding 2: Southern and Central hotspots have a mix of all price ranges, indicating a diverse property market that may be harder to predict. Conversely the eastern and northeastern areas, show low diversity containing mainly mid-range properties (green and yellow markers).

```

In [33]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from PIL import Image

sns.set(style="whitegrid")

# Creating a figure with two subplots
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(24, 12)) # Adjust

# For town vs resale_price, using the first subplot
town_mean_prices = data.groupby('town')['resale_price'].mean().sort_values()
sns.barplot(x=town_mean_prices.values, y=town_mean_prices.index, ax=axes[0])
axes[0].set_title('Mean Resale Price by Town')
axes[0].set_xlabel('Mean Resale Price')
axes[0].set_ylabel('Town')
axes[0].tick_params(axis='y', rotation=0) # Rotate y-ticks when horizontal

# Second subplot for the image
img_path = 'community-development-councils-map-of-singapore.png'
img = Image.open(img_path)
axes[1].imshow(img)
axes[1].axis('off') # Hide the axis

# Adjusting the layout
plt.tight_layout()
plt.show()

# Get the highest and lowest average resale price by town
highest_avg_price = town_mean_prices.max()
lowest_avg_price = town_mean_prices.min()

# Get the town names for the highest and lowest average resale price
town_highest_price = town_mean_prices.idxmax()
town_lowest_price = town_mean_prices.idxmin()

# Print the highest and lowest values with their corresponding town names
print(f"The highest average resale price is in {town_highest_price} with a value of {highest_avg_price}")
print(f"The lowest average resale price is in {town_lowest_price} with a value of {lowest_avg_price}")

```

Out[33]: <AxesSubplot:ylabel='town'>

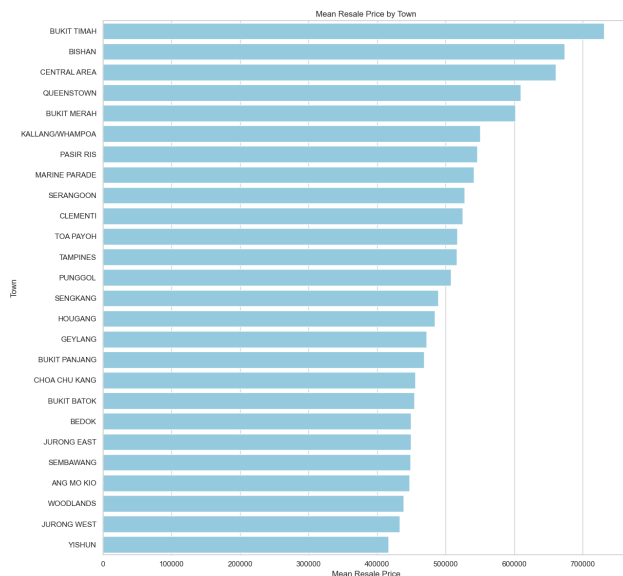
Out[33]: Text(0.5, 1.0, 'Mean Resale Price by Town')

Out[33]: Text(0.5, 0, 'Mean Resale Price')

Out[33]: Text(0, 0.5, 'Town')

Out[33]: <matplotlib.image.AxesImage at 0x7fd6f70b88e0>

Out[33]: (-0.5, 2369.5, 1535.5, -0.5)



The highest average resale price is in BUKIT TIMAH with a value of 73123.88

The lowest average resale price is in YISHUN with a value of 41652.70

Finding 3: The average property price in the most expensive town (Bukit Timah) is around 75% more expensive than the least expensive town (Yishun)

Finding 4: The top 6 most expensive towns being located in the Central Region suggests it to be the most expensive region out of the 5.

Prompted us to investigate the correlation between regions and property price by creating a new column "Region"

```
In [34]: import pandas as pd

# Create the first table with regions and towns
region_data = {
    'Region': ['West Region', 'Central Region', 'East Region', 'North-
    'Towns': [
        ['TUAS', 'PIONEER', 'JURONG WEST', 'BOON LAY', 'JURONG EAST', 'CL
        ['QUEENSTOWN', 'BUKIT TIMAH', 'TANGLIN', 'NOVENA', 'TOA PAYOH', 'BI
        ['BEDOK', 'TAMPINES', 'PAYA LEBAR', 'PASIR RIS', 'CHANGI'],
        ['ANG MO KIO', 'SERANGOON', 'HOUGANG', 'SENGKANG', 'PUNGGOL', 'SEL
        ['WOODLANDS', 'MANDAI', 'YISHUN', 'SEMBAWANG', 'SIMPANG']
    ]
}
region_df = pd.DataFrame(region_data)

# Explode the Towns list into separate rows
region_df = region_df.explode('Towns').reset_index(drop=True)

# Merge the two tables
merged_df = pd.merge(data, region_df, left_on='town', right_on='Towns')

# Drop the extra 'Towns' column
merged_df = merged_df.drop(columns=['Towns'])
```

```
In [35]: region_avg_price = merged_df.groupby('Region')['resale_price'].mean()

colors = {'West Region': 'firebrick', # light red
          'Central Region': 'khaki', # lime yellow
          'North-east Region': 'gold', # strong yellow
          'East Region': 'lightpink', # pink
          'North Region': 'orange'} # orange

# Create a bar chart
plt.figure(figsize=(10, 6))
region_avg_price.plot(kind='bar', color=[colors[region] for region in

plt.title('Average Resale Price by Region')
plt.xlabel('Region')
plt.ylabel('Average Resale Price')
plt.xticks(rotation=45)
plt.show()

# Get the highest and lowest average resale price by region
highest_avg_price = region_avg_price.max()
lowest_avg_price = region_avg_price.min()

# Get the region names for the highest and lowest average resale price
region_highest_price = region_avg_price.idxmax()
region_lowest_price = region_avg_price.idxmin()

# Print the highest and lowest values with their corresponding region
print(f"The highest average resale price is in the {region_highest_price}")
print(f"The lowest average resale price is in the {region_lowest_price}")
```

Out[35]: <Figure size 1000x600 with 0 Axes>

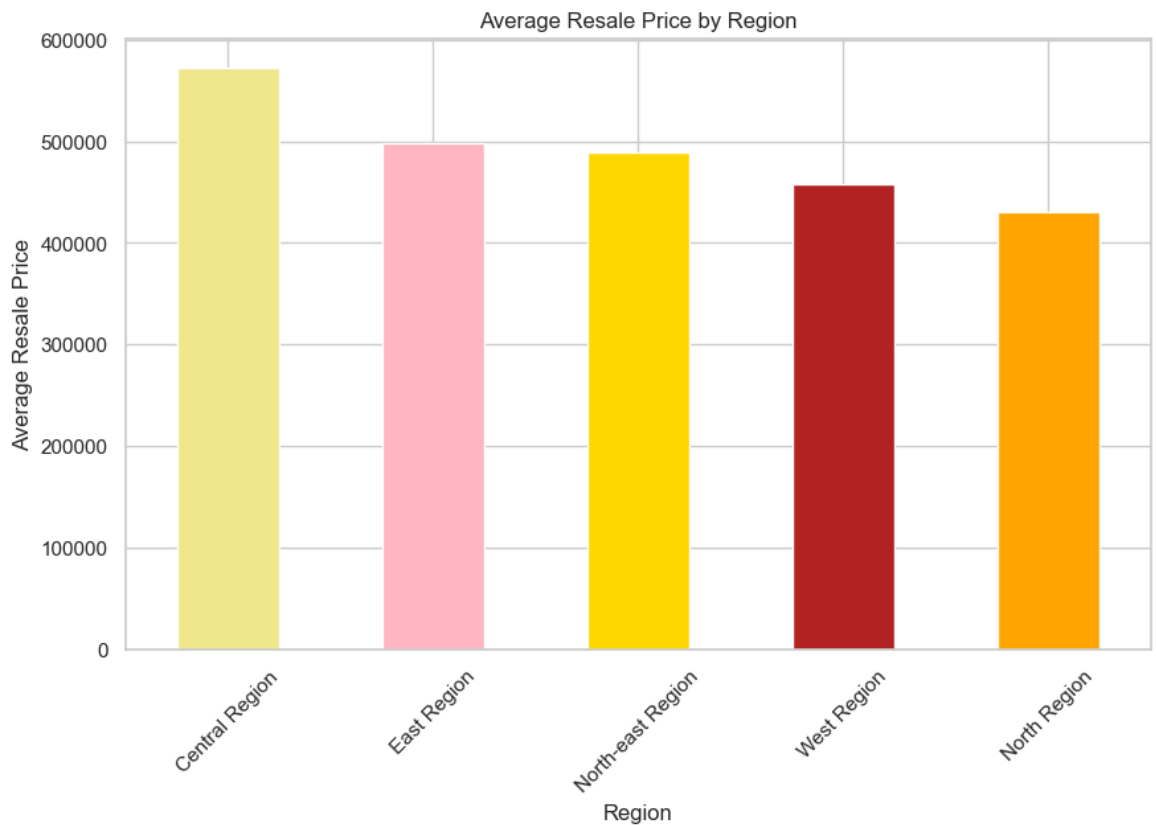
Out[35]: <AxesSubplot:xlabel='Region'>

Out[35]: Text(0.5, 1.0, 'Average Resale Price by Region')

Out[35]: Text(0.5, 0, 'Region')

Out[35]: Text(0, 0.5, 'Average Resale Price')

Out[35]: (array([0, 1, 2, 3, 4]),
 [Text(0, 0, 'Central Region'),
 Text(1, 0, 'East Region'),
 Text(2, 0, 'North-east Region'),
 Text(3, 0, 'West Region'),
 Text(4, 0, 'North Region')])



The highest average resale price is in the Central Region with a value of 572944.22

The lowest average resale price is in the North Region with a value of 431199.46

Finding 5: Confirmed the Central Region was the most expensive region with the highest average resale price

Properties in the central region were on average roughly SGD \$140,000 more expensive than properties in the least expensive region (North)

Section 5: Prediction Models

Perform one-hot encoding on categorical features with a limited number of unique values (such as town, flat_type, flat_model, Nearest_MRT_STN_NAME, Nearest_MRT_COLOR).

Ignore features like block, street_name, and full address, which may have excessively high dimensions.

Convert the 'month' feature into a date format, and potentially extract additional information from it (such as the year and month).

```
In [47]: from sklearn.preprocessing import OneHotEncoder
import numpy as np

# Convert 'month' to datetime and extract year and month as separate features
data['month'] = pd.to_datetime(data['month'])
data['year'] = data['month'].dt.year
data['month'] = data['month'].dt.month

# Selecting categorical columns for one-hot encoding
categorical_columns = ['town', 'flat_model', 'Nearest_MRT_STN_NAME', 'Nearest_MRT_STN_DISTANCE']

# Applying one-hot encoding
encoder = OneHotEncoder(sparse=False)
encoded_categorical_data = encoder.fit_transform(data[categorical_columns])

# Creating a DataFrame from the encoded categorical data
encoded_categorical_df = pd.DataFrame(encoded_categorical_data,
                                      columns=encoder.get_feature_names_out(categorical_columns))

# Dropping the original categorical columns and concatenating the encoded data
data_processed = data.drop(categorical_columns, axis=1)
data_processed = pd.concat([data_processed, encoded_categorical_df], axis=1)

# Dropping high cardinality or less relevant features
data_processed = data_processed.drop(['block', 'street_name', 'full_address'], axis=1)

# Handling NaN or infinite values
data_processed.replace([np.inf, -np.inf], np.nan, inplace=True)
data_processed.dropna(inplace=True)

# Display the processed data
data_processed.head()
```

Out [47]:

	month	flat_type	storey_range	floor_area_sqm	lease_commence_date	remaining_lease	resale_price
0	1.0	2.0	11.0	44.0	1979.0	732.0	133251
1	1.0	3.0	2.0	67.0	1978.0	720.0	33313
2	1.0	3.0	2.0	67.0	1980.0	744.0	133251
3	1.0	3.0	5.0	68.0	1980.0	745.0	33313
4	1.0	3.0	2.0	67.0	1980.0	744.0	133251

```
In [48]: from sklearn.model_selection import train_test_split, cross_val_score

# Separating the target variable and features
X = data_processed.drop('resale_price', axis=1)
y = data_processed['resale_price']

# Splitting the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Displaying the shapes of the training and testing sets
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Out [48]: ((133251, 198), (33313, 198), (133251,), (33313,))

5.1 Linear Regression Model

```
In [49]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

# Evaluating the Linear Regression model
mae_linear = mean_absolute_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mean_squared_error(y_test, y_pred_linear))
r2_linear = r2_score(y_test, y_pred_linear)

# Cross validation
cv_scores_linear = cross_val_score(linear_model, X, y, cv=5, scoring='r2')
cv_rmse_linear = np.sqrt(-cv_scores_linear).mean()

print("Linear MAE:", mae_linear)
print("Linear RMSE:", rmse_linear)
print("Linear R2:", r2_linear)
print("Linear cv RMSE :", cv_rmse_linear)
```

```
Out[49]: LinearRegression()

Linear MAE: 55209.445496375
Linear RMSE): 72656.10609000582
Linear R2: 0.8143094707403571
Linear cv RMSE : 82434.49812364359
```

```
In [50]: Coeff = pd.DataFrame({
    'variable': X_train.columns,
    'coefficient': linear_model.coef_,
    'intercept': linear_model.intercept_
})

Coeff.iloc[Coeff['coefficient'].abs().sort_values(ascending=False).ind
```

Out [50]:

		variable	coefficient	intercept
6		LATITUDE	-2.173367e+06	-6.390659e+07
7		LONGITUDE	1.516309e+05	-6.390659e+07
16		town_BUKIT MERAH	-1.452367e+05	-6.390659e+07
125	Nearest_MRT_STN_NAME_LABRADOR PARK MRT STATION		1.289649e+05	-6.390659e+07
129	Nearest_MRT_STN_NAME_LITTLE INDIA MRT STATION		1.182289e+05	-6.390659e+07
...		...	...	...
69	Nearest_MRT_STN_NAME_BENCOOLEN MRT STATION		1.332743e+03	-6.390659e+07
168	Nearest_MRT_STN_NAME_SOUTH VIEW LRT STATION		-1.103953e+03	-6.390659e+07
17		town_BUKIT PANJANG	4.329191e+02	-6.390659e+07
11		year	-4.049798e-08	-6.390659e+07
0		month	1.018232e-08	-6.390659e+07

198 rows × 3 columns

From the perspective of features with a larger proportion of coefficients, the majority are related to geography and the type of housing. This indicates that the location and type of a house are very important. It also reflects that, in addition to numerical features, some text features related to the target variable are also very important for model prediction. Conversely, features related to time are not as significant.

5.2 Gradient Boosting Regression Trees

```
In [51]: from sklearn.ensemble import GradientBoostingRegressor

# Training the Gradient Boosting Regressor model
gb_model = GradientBoostingRegressor(random_state=20, learning_rate=0.2)
gb_model.fit(X_train, y_train)

# Predicting on the test set
y_pred_gb = gb_model.predict(X_test)

# Evaluating the Gradient Boosting Regressor model
mae_gb = mean_absolute_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for Gradient Boosting Regressor
cv_scores_gb = cross_val_score(gb_model, X, y, cv=5, scoring='neg_mean_squared_error')
cv_rmse_gb = np.sqrt(-cv_scores_gb).mean()

print("Linear MAE:", mae_gb)
print("Linear RMSE:", rmse_gb)
print("Linear R2:", r2_gb)
print("Linear cv RMSE:", cv_rmse_gb)
```

```
Out[51]: GradientBoostingRegressor(learning_rate=0.2, max_depth=5, random_state=20)
```

```
Linear MAE: 28921.057991271966
Linear RMSE: 38619.94816762523
Linear R2: 0.9475350951210988
Linear cv RMSE: 49864.47549247596
```

Section 6: Comparison and Analysis

Comparison and Analysis:

Performance: Gradient Boosting Regression Trees outperform Linear Regression on all metrics, showing lower MAE and RMSE, as well as a higher R^2 value. Model Complexity: Gradient Boosting Regression Trees are more complex than Linear Regression, requiring more computational resources and time for training and tuning. Interpretability: Linear Regression models are generally easier to interpret as they provide direct coefficients that describe the relationship between features and the target. Gradient Boosting Regression Trees, on the other hand, are more difficult to interpret as they are ensemble models based on a large number of decision trees. Model Limitations:

Linear Regression:

Sensitive to outliers. Assumes a linear relationship between features and the target, which may not always be the case. Gradient Boosting Regression Trees:

More prone to overfitting, especially in cases of insufficient data or too many features. Model training and tuning can be more complex and time-consuming. Poor interpretability of results.

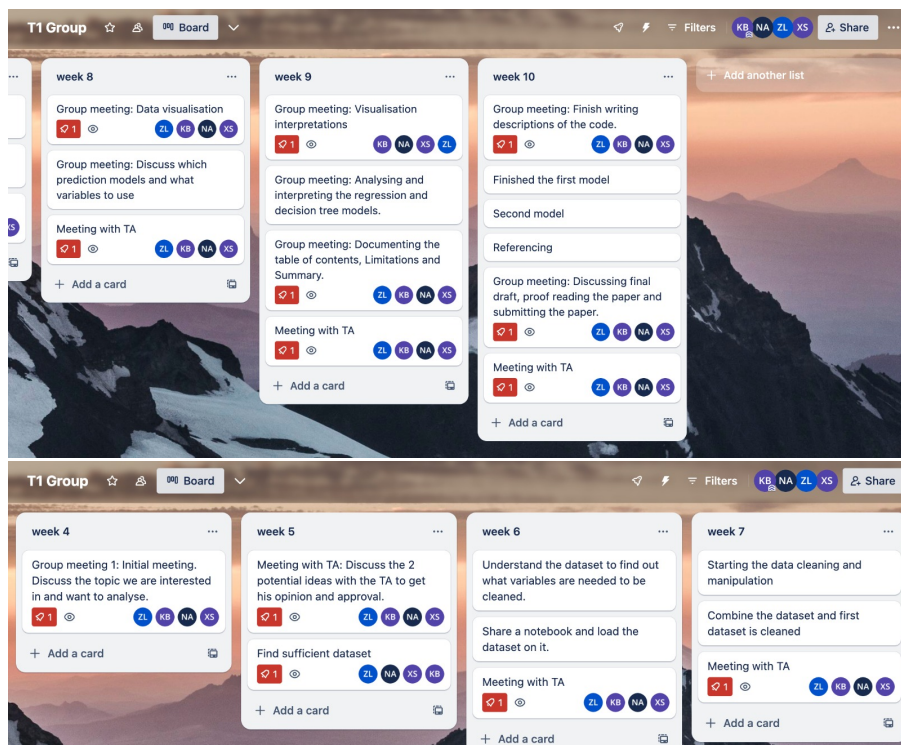
Section 7: Appendices

Trello Board

```
In [52]: from IPython.display import display, HTML

image_paths = ["trello week4-7.png", "trello week8-10.png"]
image_html = "".join(
    f'<img src="{image_path}" style="display:inline-block; margin: 1px'
    for image_path in image_paths
)

display(HTML(image_html))
```



Meeting Minutes (26/10 - 7/12)

```
In [102]: from PIL import Image
from IPython.display import display

image_paths = [
    "Meeting Minutes 1.png",
    "Meeting Minutes 2.png",
    "Meeting Minutes 3.png",
    "Meeting Minutes 4.png",
    "Meeting Minutes 5.png",
    "Meeting Minutes 6.png",
    "Meeting Minutes 7.png",
    "Meeting Minutes 8.png",
]

images = [Image.open(image_path) for image_path in image_paths]

common_size = (450, 500)
resized_images = [image.resize(common_size) for image in images]
canvas_width = common_size[0] * 4
canvas_height = common_size[1] * 2
canvas = Image.new("RGB", (canvas_width, canvas_height))

for i, image in enumerate(resized_images):
    x = (i % 4) * common_size[0]
    y = (i // 4) * common_size[1]
    canvas.paste(image, (x, y))

# Display the canvas image in the notebook
display(canvas)
```

Group meeting 1: meeting minutes Location: Canada Sq (City Level 50 BNSF Lecture Theatre) Date: 26/10/2023 Time: 19:30-20:30 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Free flow sharing of ideas from group members - Discuss potential/interest areas and topics - Discuss the merits and feasibility of each topic <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Research and compile potential topics</td><td>AI</td><td>21/10/2023</td><td>Complete</td></tr><tr><td>Filter topics into 2-3 topics with highest potential</td><td>AI</td><td>21/10/2023</td><td>Complete</td></tr><tr><td>Familiarise the coursework brief and structure</td><td>AI</td><td>21/10/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Research and compile potential topics	AI	21/10/2023	Complete	Filter topics into 2-3 topics with highest potential	AI	21/10/2023	Complete	Familiarise the coursework brief and structure	AI	21/10/2023	Complete	Group meeting 2: meeting minutes Location: Microsoft Teams (Online) Date: 31/10/2023 Time: 10:00-11:00 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Discuss shared (2-3) potential topics with TA - Discuss appropriate data sources and gathering methods - Clarify coursework brief and objectives with TA <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Compare both topics</td><td>AI</td><td>9/11/2023</td><td>Complete</td></tr><tr><td>Research potential datasets for both topics</td><td>AI</td><td>9/11/2023</td><td>Complete</td></tr><tr><td>Make group of interest and compare strength and feasibility of each (for both topics)</td><td>AI</td><td>9/11/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Compare both topics	AI	9/11/2023	Complete	Research potential datasets for both topics	AI	9/11/2023	Complete	Make group of interest and compare strength and feasibility of each (for both topics)	AI	9/11/2023	Complete	Group meeting 3: meeting minutes Location: Microsoft Teams (Online) Date: 31/10/2023 Time: 10:00-11:00 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Brief overview from TA on chosen topics - Present selected datasets to TA and seek approval - Clarify coursework brief and objectives with TA - Identify potential areas of analysis and methods <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Kai Berrington: research dataset, identify variables (dataset)</td><td>Kai Berrington</td><td>16/11/2023</td><td>Complete</td></tr><tr><td>Ziqian Li: Create notebook (dataset) for the group</td><td>Ziqian Li</td><td>16/11/2023</td><td>Complete</td></tr><tr><td>Xinyi Shen: Upload and run dataset in notebook</td><td>Xinyi Shen</td><td>16/11/2023</td><td>Complete</td></tr><tr><td>Northgal Apeaspath: Identify variables, determine if to be cleaned</td><td>Northgal Apeaspath</td><td>16/11/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Kai Berrington: research dataset, identify variables (dataset)	Kai Berrington	16/11/2023	Complete	Ziqian Li: Create notebook (dataset) for the group	Ziqian Li	16/11/2023	Complete	Xinyi Shen: Upload and run dataset in notebook	Xinyi Shen	16/11/2023	Complete	Northgal Apeaspath: Identify variables, determine if to be cleaned	Northgal Apeaspath	16/11/2023	Complete	Group meeting 4: meeting minutes Location: Microsoft Teams (Online) Date: 31/10/2023 Time: 10:00-11:00 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Present final chosen datasets to TA - Update on data analysis and cleaning approaches - Finalise variables (columns to be deleted) <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Data Cleaning for all selected variables</td><td>Northgal Apeaspath</td><td>23/11/2023</td><td>Complete</td></tr><tr><td>Combine all datasets</td><td>Kai Berrington</td><td>23/11/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Data Cleaning for all selected variables	Northgal Apeaspath	23/11/2023	Complete	Combine all datasets	Kai Berrington	23/11/2023	Complete				
Action Items	Owner(s)	Deadline	Status																																																																				
Research and compile potential topics	AI	21/10/2023	Complete																																																																				
Filter topics into 2-3 topics with highest potential	AI	21/10/2023	Complete																																																																				
Familiarise the coursework brief and structure	AI	21/10/2023	Complete																																																																				
Action Items	Owner(s)	Deadline	Status																																																																				
Compare both topics	AI	9/11/2023	Complete																																																																				
Research potential datasets for both topics	AI	9/11/2023	Complete																																																																				
Make group of interest and compare strength and feasibility of each (for both topics)	AI	9/11/2023	Complete																																																																				
Action Items	Owner(s)	Deadline	Status																																																																				
Kai Berrington: research dataset, identify variables (dataset)	Kai Berrington	16/11/2023	Complete																																																																				
Ziqian Li: Create notebook (dataset) for the group	Ziqian Li	16/11/2023	Complete																																																																				
Xinyi Shen: Upload and run dataset in notebook	Xinyi Shen	16/11/2023	Complete																																																																				
Northgal Apeaspath: Identify variables, determine if to be cleaned	Northgal Apeaspath	16/11/2023	Complete																																																																				
Action Items	Owner(s)	Deadline	Status																																																																				
Data Cleaning for all selected variables	Northgal Apeaspath	23/11/2023	Complete																																																																				
Combine all datasets	Kai Berrington	23/11/2023	Complete																																																																				
Group meeting 5: meeting minutes Location: Microsoft Teams (Online) Date: 23/11/2023 Time: 10:00-11:00 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Present cleaned data to TA - Clarify with TA on coursework requirements on data cleaning and analysis - Discuss potential analysis and visualisation <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Generate data visualisation</td><td>Xinyi Shen, Ziqian Li</td><td>30/11/2023</td><td>Complete</td></tr><tr><td>Draft final notebook submission (all members to be)</td><td>Kai Berrington, Northgal Apeaspath</td><td>30/11/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Generate data visualisation	Xinyi Shen, Ziqian Li	30/11/2023	Complete	Draft final notebook submission (all members to be)	Kai Berrington, Northgal Apeaspath	30/11/2023	Complete	Group meeting 6: meeting minutes Location: Microsoft Teams (Online) Date: 30/11/2023 Time: 10:00-11:00 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Present all data visualisations to TA - Discuss on prediction models and application towards Impregnation flat price - Clarify coursework brief and objectives with TA, data analysis and visualisations <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Draft findings and insights from model</td><td>AI</td><td>7/12/2023</td><td>Complete</td></tr><tr><td>Research and attempt draft and required writing</td><td>AI</td><td>7/12/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Draft findings and insights from model	AI	7/12/2023	Complete	Research and attempt draft and required writing	AI	7/12/2023	Complete	Group meeting 7: meeting minutes Location: Microsoft Teams (Online) Date: 3/12/2023 Time: 10:00-11:00 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Finalise analysis and findings, present to TA - Discuss prediction models on regression and decision tree models - Finalise and require data visualisation and map (if any) <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>(Continued) Construct prediction model</td><td>Xinyi Shen, Ziqian Li</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Work on notebook</td><td>Kai Berrington, Northgal Apeaspath</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Finalise all related files to be used for final submission</td><td>AI</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Complete data analysis and findings</td><td>AI</td><td>8/12/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	(Continued) Construct prediction model	Xinyi Shen, Ziqian Li	8/12/2023	Complete	Work on notebook	Kai Berrington, Northgal Apeaspath	8/12/2023	Complete	Finalise all related files to be used for final submission	AI	8/12/2023	Complete	Complete data analysis and findings	AI	8/12/2023	Complete	Group meeting 8: meeting minutes Location: Canada Sq (City Level 50 BNSF Lecture Theatre) Date: 7/12/2023 Time: 14:00-16:30 Attendees: Kai Berrington, Northgal Apeaspath, Xinyi Shen, Ziqian Li Teaching Assistant: Karri Tytani Agenda items - Finalise prediction models on regression and decision tree models - Finalise analysis and findings, and related texts - Reprint each group member on their assigned sections - Discuss limitations and potential project outcomes <table><thead><tr><th>Action Items</th><th>Owner(s)</th><th>Deadline</th><th>Status</th></tr></thead><tbody><tr><td>Draft notebook and upload into IPYNB</td><td>AI</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Reprint</td><td>AI</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Continue building prediction model</td><td>Xinyi Shen, Ziqian Li</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Complete references and citations</td><td>AI</td><td>8/12/2023</td><td>Complete</td></tr><tr><td>Read final draft and proof reading</td><td>AI</td><td>8/12/2023</td><td>Complete</td></tr></tbody></table>	Action Items	Owner(s)	Deadline	Status	Draft notebook and upload into IPYNB	AI	8/12/2023	Complete	Reprint	AI	8/12/2023	Complete	Continue building prediction model	Xinyi Shen, Ziqian Li	8/12/2023	Complete	Complete references and citations	AI	8/12/2023	Complete	Read final draft and proof reading	AI	8/12/2023	Complete
Action Items	Owner(s)	Deadline	Status																																																																				
Generate data visualisation	Xinyi Shen, Ziqian Li	30/11/2023	Complete																																																																				
Draft final notebook submission (all members to be)	Kai Berrington, Northgal Apeaspath	30/11/2023	Complete																																																																				
Action Items	Owner(s)	Deadline	Status																																																																				
Draft findings and insights from model	AI	7/12/2023	Complete																																																																				
Research and attempt draft and required writing	AI	7/12/2023	Complete																																																																				
Action Items	Owner(s)	Deadline	Status																																																																				
(Continued) Construct prediction model	Xinyi Shen, Ziqian Li	8/12/2023	Complete																																																																				
Work on notebook	Kai Berrington, Northgal Apeaspath	8/12/2023	Complete																																																																				
Finalise all related files to be used for final submission	AI	8/12/2023	Complete																																																																				
Complete data analysis and findings	AI	8/12/2023	Complete																																																																				
Action Items	Owner(s)	Deadline	Status																																																																				
Draft notebook and upload into IPYNB	AI	8/12/2023	Complete																																																																				
Reprint	AI	8/12/2023	Complete																																																																				
Continue building prediction model	Xinyi Shen, Ziqian Li	8/12/2023	Complete																																																																				
Complete references and citations	AI	8/12/2023	Complete																																																																				
Read final draft and proof reading	AI	8/12/2023	Complete																																																																				

Member's Contribution

In [93]: `from IPython.display import Image
Image("Member's Contribution.png")`

Out [93]:

MSIN0143 Programming for Business Analytics Group Coursework

Group member's name	Contribution
Kai Bermingham	• Data preparation; data description • Data visualization • Introduction • Data analysis and findings • Communication and updates with TA
Nonthapat Arparpardh	• Data preparation; data dictionary • Data cleaning • Data analysis and findings • Meeting minutes, project management • Communication and updates with TA
Xinyu Shen	• Data preparation • Data visualization • Data analysis and findings • Prediction models and comparison • Communication and updates with TA
Ziqian Li	• Data preparation • Data visualization • Data analysis and findings • Trello board, project management • Communication and updates with TA

Section 8: References

- Chua, H.X. (2020) mrtsg.csv, GitHub Repository. Available at: <https://github.com/hxchua/datadoubleconfirm/blob/master/datasets/mrtsg.csv> (<https://github.com/hxchua/datadoubleconfirm/blob/master/datasets/mrtsg.csv>) (Accessed: 11 December 2023).
- Housing & Development Board (2023) HDB | about Us - Housing & Development Board, HDB History and Towns. Available at: <https://www.hdb.gov.sg/about-us> (<https://www.hdb.gov.sg/about-us>) (Accessed: 08 December 2023).
- Housing & Development Board (2023) Resale flat prices based on registration date from Jan-2017 onwards, Data.gov.sg. Available at: https://beta.data.gov.sg/datasets/d_8b84c4ee58e3cfc0ece0d773c8ca6abc/view (https://beta.data.gov.sg/datasets/d_8b84c4ee58e3cfc0ece0d773c8ca6abc/view) (Accessed: 11 December 2023).
- Olszewski, P. and Wibowo, S.S. (2005) 'Using equivalent walking distance to assess pedestrian accessibility to transit stations in Singapore', Transportation Research Record: Journal of the Transportation Research Board, 1927(1), pp. 38–45. doi:10.1177/0361198105192700105.

- Shui, J. and Murthy, S. (2019) 'Under what circumstances do first-time homebuyers overpay? — an empirical analysis using mortgage and appraisal data', Journal of Real Estate Research, 41(1), pp. 107–146. doi:10.1080/10835547.2019.12091518.
- Singapore Food Agency (2019) Supermarkets (GEOJSON), Data.gov.sg. Available at: https://beta.data.gov.sg/datasets/d_cac2c32f01960a3ad7202a99c27268a0/view (https://beta.data.gov.sg/datasets/d_cac2c32f01960a3ad7202a99c27268a0/view). (Accessed: 11 December 2023).
- Sng, G. (2022) 9 most expensive neighbourhoods in Singapore for a dose of serious house envy, Uchify. Available at: <https://uchify.com/expensive-neighbourhoods-sg/> (<https://uchify.com/expensive-neighbourhoods-sg/>). (Accessed: 09 December 2023).
- Yue, C.S. (2023) Investigating Singapore's public housing issues, East Asia Forum. Available at: <https://www.eastasiaforum.org/2023/04/15/investigating-singapores-public-housing-issues/#:~:text=An%20indicator%20of%20housing%20affordability,that%20housing%20> (<https://www.eastasiaforum.org/2023/04/15/investigating-singapores-public-housing-issues/#:~:text=An%20indicator%20of%20housing%20affordability,that%20housing%20>). (Accessed: 08 December 2023).

In []: